



## 11. Java Methods

### Exam Objectives

1. Describe and create a method
2. Create and use accessor and mutator methods
3. Create overloaded methods
4. Describe a static method and demonstrate its use within a program

## 11.1 Create methods with arguments and return values

### 11.1.1 Creating a method

We have already seen methods in previous chapters. Indeed, we have been using the "main" method to run our test programs all along. But we haven't discussed them formally yet.

A **method** is what gives **behavior** to a type. Java allows classes, interfaces, and enums to have a behavior. A behavior is nothing but a high level action performed by a piece of code. If you expect this action to be performed as and when required, you put this piece of code in a method and give it a name. This lets you invoke that action whenever you need by using the name. It is like a black box to which you give some input and get back some output in return.

Thus, the basic structure of a method is as follows:

```
returnType methodName(parameters) {  
    methodBody  
}
```

A **ReturnType** specifies what the method returns as a result of its execution. For example, a method that returns the sum of two integers may specify that it returns an `int` as a result. A method that returns an `Account` object may specify `Account` as the return type. If a method doesn't return anything, it must specify so, using the keyword `void`.

It is important to know that a method can return one thing at the most.

A **MethodName** is the name given to this method. It must be a valid Java identifier. Remember that an identifier is an unlimited-length sequence of Java letters and Java digits, the first of which must be a Java letter. It cannot be a reserved word or a keyword.

The **Parameters** of a method are specified by a list of comma-separated parameter declarations. Each parameter declaration consists of a type and an identifier that specifies the name of the parameter. If a method does not take any parameter, the parameters list will be empty. The following are a few examples of a valid parameter list:

```
void save() //no parameters  
void saveAccount(Account acct) //takes an Account as a parameter  
void add(int a, int b) //takes two ints as parameters
```

Note that, unlike regular declarations, each parameter in a parameter list must be specified along with its type individually. Thus, while `int a, b;` is a valid statement on its own that declares two `int` variables, `add(int a, b)` is an invalid declaration of method parameter `b`.

Each parameter in the parameter list may also be declared as `final` if the code in the method does not change its value. For example:

```
void add(int a, final int b) //b is final
```

If a parameter is declared `final` and if the method body tries to change its value, the code will fail to compile.

The **MethodBody** contains the code that is to be executed upon invocation of that method. It must be contained within curly braces. Observe that this is unlike other block statements such as `if/else`, `for`, and `while`, where you can omit the curly braces if there is just one statement in their body. In the case of methods, curly braces are required even if the body consists of just one statement.

There are several other bells and whistles associated with methods such as `var-args`, `accessibility`, `static`, `abstract`, `final`, and `exception handling`. I will discuss all of these as we go along.

### 11.1.2 Returning a value from a method

A method must always return a value of the type that it promises to return in its declaration after successful completion. This means that it is not possible to return a value conditionally as shown in the following code:

```
public int get2X(int x){  
    if(x>0) return 2*x;  
}
```

The compiler will generate an error message saying, "error: missing return statement" because it notices that the method will not return anything if the `if` condition is false. This makes sense if you consider what will happen to the caller of the `get2X` method if the method doesn't return anything. For example, what value should be assigned to `y` by this statement - `int y = get2X(-1);` ? There is no good answer. Thus, the method must return an `int` in all situations (except when it throws an exception, but you can ignore that for now).

This rule applies even to methods that return a reference type (and not just to method that return a primitive). But if the return type of a method is a reference type, it is ok for the method to return `null` because `null` is a valid value for a reference. In other words, returning `null` is not the same as returning nothing. Thus, the following method is fine:

```
String getValue(int x){  
    if(x > 0) return "good day!";  
    else return null; //this is ok  
}
```

The only situation where a method can avoid returning a value is if it ends up throwing an exception, which means that the method didn't really finish successfully and therefore, it cannot be expected to return a value!

#### Method returning void

If a method says that it doesn't return anything (i.e. its return type is specified as `void`), then it must not return anything in any situation. Obviously, it cannot even return `null` because as we

saw above, `null` is not the same as nothing. But the interesting thing is that it cannot even return `void`. Thus, the following method will not compile.

```
void doSomething(){
    System.out.println("hello");
    return void; //invalid
}
```

There are only two options in this case - do not have any return statement at all or have an empty return statement, i.e., `return;` For example:

```
void doSomething(){
    System.out.println("hello");
    return; //empty return
}
```

or

```
void doSomething(){
    System.out.println("hello");
    //no return statement at all
}
```

## Returning values of different types from a method

The general rule is that Java does not allow a method to return a value that is of a different type than the one specified in its declaration. This means that if a method says that it returns an `int`, it cannot then return a `boolean` value. However, there are three exceptions to this rule. Two are about primitives and one is about references.

1. **Numeric promotion** - If the return type of a method is a numeric type (i.e. `byte`, `char`, `short`, `int`, `long`, `float`, and `double`), then the return value can be of any other numeric type as long as the type of the return value is smaller than the type of the declared return type. For example, if a method says it returns an `int`, it is ok for that method to return a `byte`, `short`, or `char` value. But it cannot return a `long`, `float`, or `double` value. Thus, the following code will compile fine:

```
public int getVal(int x){
    char ch = 'a';
    byte b = 0;
    if(x>0) return ch;
    else return b;
}
```

This is allowed because a smaller type can easily be promoted to a larger type without any loss of information.

2. **Autoboxing/Unboxing** - Java allows a return value to be a reference to a wrapper class if the return type is of a primitive type of the same or a larger type. Thus, the following code is ok:

```
public int getVal(){
    return new Short((short)10); //wrapper object will be unboxed into a
    primitive
    //return new Long(10); //will not compile, Long cannot be converted to int
}
```

The reverse is also allowed but only when the method's return type is a wrapper of the same type as the type of the primitive value being returned:

```
public Integer getVal(){
    byte b = 10;
    return 10; //ok, int 10 will be boxed into an Integer object
    //return b; //will not compile, byte cannot be converted to Integer
}
```

3. The third exception is related to inheritance. Since I haven't discussed this topic yet, I will only mention it briefly here. Don't worry if you don't understand it completely at this point. It is ok for a method to return a reference of a subtype of the type declared as its return type. This is called "covariant return types." For example, if a method declares that it returns `Object`, it is ok for the method to return a `String` because a `String` is also an `Object`. The following code shows some valid possibilities:

```
Object getValue(){
    return "hello";
    //return 10; //This is ok, 10 will be boxed into an Integer object, which is an
    Object
    //return null; //This is ok too
}
```

This is the same as promising someone that you will give them a fruit, and then give them a banana. This is ok because a banana is also a type of fruit. But the reverse is not true. If you promise someone that you will give them a banana, you cannot then give any other fruit. Thus, the following will fail to compile:

```
String getValue(){
    return new Object(); // will not compile because an Object is not a String
}
```

It will become clearer after you learn about inheritance.

These rules ensure that only those values that are "assignable" to the declared return type are returned by a method. In other words, if you can assign a particular value to a variable of the declared return type directly, then you can return that value from that method. For example, you can assign a `char` value to an `int` variable directly, therefore, you can return a `char` value from a method that declares that it returns an `int`.

## Returning multiple values from a method

Java does not allow a method to return more than one value. This seems like difficult restriction to overcome if you want to return multiple values from a method. For example, what if your `getName` method wants to return first name and last name separately? Well, the way to do that in Java is to use a class to capture multiple values and then return a reference to an object of that class. Here is what your `getName` method may look like:

```
Name getName(){
    Name n = new Name(); //capture two values in a Name object
    n.firstName = "Ann";
    n.lastName = "Rand";
    return n; //return a reference to the Name object
}
```

This code assumes the existence of a separate class called `Name` that can capture the two components of the name:

```
class Name{
    String firstName, lastName;
}
```

You may also use arrays to overcome this restriction. For example, the `getName` method can also be coded as follows, but it is a hack and a bad practice:

```
String[] getName(){
    return new String[]{"ann", "rand" };
}
```

Generally, classes are designed to capture values that are related to each other. For example, you might have a `Student` class that captures a student's id, name, and address. You may, however, encounter situations where you want to return multiple unrelated values from the same method. You can use the same approach to return these values. Classes that are used to capture unrelated values are called holder classes. If you encounter such situations too often in your code then it is a symptom of a bad design. A detailed discussion on this is beyond the scope of the exam.

### 11.1.3 Varargs

If you know the type of the arguments but don't know the exact number of arguments that the caller is going to pass to a method, you can put the arguments in an array and pass the array to the method. For example, if you want to write a method that computes the average of any number of integer values, you could do something like this:

```
public double average(int[] values){
    /* by the way, can you tell what will happen if sum is declared as int?
       Expect questions in the exam that seem to be about one topic
```

```
        but are actually about something entirely different.
    */
    double sum = 0;
    for(int i=0; i<values.length; i++) sum += values[i];
    return values.length==0?0 : sum/values.length;
}
```

The caller of this method can put all the integer values in an array and call it as follows:

```
int[] values = { 1, 2, 3, 4 };
double average = average(values);
```

This approach works fine but is a little tedious to write. Java 5 introduced a new syntax called "**varargs**" that makes passing a variable number of arguments to a method a little easier. Instead of using an array parameter in the method declaration, you use a varargs parameter by appending three dots to int like this:

```
public double average(int... values){
    //same code as shown earlier goes here
}
```

There is absolutely no change in the method body. Within the method body, `values` remains an array of integers like before. The caller, however, does not need to create an array explicitly. It can simply pass any number of int arguments in the method invocation. Thus, the following are all valid method invocations of the new average method.

```
double average = average(); //no argument
double average = average(1); //one argument
double average = average(1, 2, 3, 4); //multiple arguments
```

An important point to understand here is that the varargs syntax is just a syntactic sugar for the developer. It saves a few keystrokes while typing the code but makes absolutely no difference to the resulting bytecode generated by the compiler. When the compiler sees the invocation of a method with varargs parameter, it simply wraps the arguments into an array and passes the array to the method. Indeed, the old code that used an array to call the average method still works with the new average method. If you update the method code to print the number of elements in the values array (just add `System.out.println(values.length);`), you will see it print 0, 1, and 4 for the above three invocations respectively.

Observe that if you don't pass any argument, the compiler will create an array with a length of zero and pass that array. The method will not receive a null but an array of length zero in this case. This is unlike the other ways that you invoke the array version of the method. It is not possible to invoke the array version without any argument. If you don't want to pass any value, you will have to pass null. Try it out and see what happens.

You can apply the varargs approach to a parameter of any type and not just primitives.

## Restrictions on varargs

Let us try to expand the usage of varargs from one parameter to two. What if we define a method as follows:

```
public double test(int... p1, int... p2){
    System.out.println(p1.length+" "+p2.length);
}
```

and call this method like as follows?

```
test(1, 2);
```

This poses a problem for the compiler. It has three equally valid possibilities for initializing p1 and p2. It can create two int arrays containing {1} and {2}, or it can create {1, 2} and { }, or it can create { } and { 1, 2 } and pass them as arguments for p1 and p2 respectively.

Resolving this ambiguity through complicated rules would make the varargs feature too confusing to use and so, Java imposes the following two restrictions on varargs:

1. A method cannot have more than one varargs parameter.
2. The varargs parameter, if present, must be the last parameter in the parameter list of a method.

With the above two rules in place, it is easy to understand what will happen if you have a method as follows:

```
void test(int x, int... y){
    //some code here
}
```

and call it as follows:

```
test(1, 2); //x is assigned 1, y is assigned {2}
test(1); //x is assigned 1, y is assigned { }
test(1, 2, 3, 4); //x is assigned 1, y is assigned { 2, 3, 4}
```

Note that since x is a non-varargs parameter, any invocation of the method test must include a value for x. Therefore, a call to test() with no argument will not compile.

## 11.2 Create overloaded methods

### 11.2.1 Method signature

Before learning about method overloading, you need to know about something called "method signature".

A **method signature** is kind of an "id" of a method. It uniquely identifies a method in a class. A class may have several methods with the same name but it cannot have more than one method with the same **signature**. When you call a method of a class, you basically tell the compiler which

method you mean to call by mentioning its signature. If a class has multiple methods with the same signature, the compiler will not be able to determine which method you mean and will raise an error.

The question then is: what exactly constitutes a method signature? Simple. Method signature includes just the **method name** and its **ordered list of parameter types**. Nothing else. For example, all of the following method declarations have the same signature:

1. `void process(int a, String str);`
2. `public void process(int value, String name);`
3. `void process(int a, String str) throws Exception;`
4. `String process(int a, String str);`
5. `private int process(int a, String data);`
6. `static void process(int a, String str);`

Observe that in all of the above cases, the method name (i.e. `process`) and the ordered list of parameter types (i.e. `int, String`) are exactly the same and their access types, static/instance types, return types, parameter names, and the throws clauses are all different. However, since these attributes are not part of the method signature, they do not make the methods different. The compiler will consider all of the above methods as having the same signature and will complain if you have any two of the above methods in the same class.

### 11.2.2 Method overloading

Now that you know about method signature, method overloading is a piece of cake. You know that a class cannot have more than one method with the same signature. But it can certainly have multiple methods with the same name and different parameter types because having different parameter types would make their signatures different. In such a situation where a class has multiple methods with same name, it is said that the class has "**overloaded**" the method name. The method name is overloaded in the sense that there are multiple possibilities associated with that name.

From the compiler's perspective, method overloading is nothing special. Since the compiler cares only about the method signatures, it makes no difference to the compiler whether two methods are different because of a difference in their method names, or their method parameters, or both. To the compiler, overloaded methods are just different methods.

However, method overloading does hold importance for the developer and the users of a class because it can either make the code more intuitive to use or make it totally confusing. For example, you have been using `System.out.println` methods to print out all sorts of values to the console. You are able to use the same method named `println` for printing an `int` as well as a `String` only because there are multiple `println` methods that take different parameter types. Take a look at the JavaDoc API description of `java.io.PrintStream` class to see how many `println` methods it has. Wouldn't you be frustrated if you had to use `printlnByte` to print a byte, `printlnShort` to print a short, `printlnString` to print a `String` and so on? In this case, method overloading has certainly made your life easier.

Now, consider the following code:

```
public class TestClass{
    static void doSomething(Integer i, short s){
        System.out.println("1");
    }

    static void doSomething(int in, Short s){
        System.out.println("1");
    }

    public static void main(String[] args){
        int b = 10;
        short x = 20;
        doSomething(b, x);
    }
}
```

Any guess on what the above code prints? Don't worry, even experienced programmers will scratch their heads while figuring this one out. The answer is that it will not compile. But not because of the presence of two `doSomething` methods. The methods are fine. They have different signatures. The problem is with the call to `doSomething(b, x)`. The compiler is not able to determine which one of the two `doSomething` methods to use because both of them are equally applicable.

You will not get questions this hard in the exam. I showed you the above code only to illustrate how overloading can be misused to create horrible code.

### 11.2.3 Method selection

The code that I showed you in the previous section illustrates a problem with overloaded methods. When their parameter lists are not too different, it is very difficult to figure out which of the overloaded methods will be picked up for a method call. If the parameters are too similar, it may even become an impossible task.

Remember that even though the actual method is selected and executed at runtime, it is the compiler's job to validate the code and make sure that a particular method call can be bound unambiguously to a specific method out of several available methods. Since actual objects are created by the JVM at run time, the compiler does not always know the exact type of the object pointed to by a reference variable. Therefore, the compiler considers only the declared type of the variable on which the method is invoked and the declared type of the variables passed as arguments while determining which method should be invoked. Once it determines the method that should be invoked, it binds the method call to that particular method signature. This part is important because if the compiler allowed any method to be called on any type of reference, the call may fail at run time. By making the compiler perform this analysis upfront at compile time, we reduce the chances of coding mistakes getting discovered at runtime. If you have written any JavaScript code, you know what I am talking about. This ties back to the strongly typed nature of the Java language.

At runtime, the JVM picks up the method signature that was selected by the compiler, finds

out the actual object pointed to by the reference variable, and invokes the method on that object. This is the part where inheritance plays a huge role and where the magic of polymorphism happens. I will talk about it in the next chapter but for now, I will just focus on overloading without the complication of inheritance.

While you will not get questions that are as complicated as shown above in the exam, you still need to know a few basic rules to figure out the simple cases. So, here they are:

1. The first rule is that having overloaded methods does not cause a compilation error by itself. In other words, as long as their method signatures are different, the compiler doesn't care whether they are too similar or not. Compilation error occurs only if the compiler is not able to successfully disambiguate a particular method call. Java specifies precise rules that are used by the compiler to disambiguate such method calls.
2. **Exact match** - If the compiler finds a method whose parameter list is an exact match to the argument list of the method call, then it selects that method. For example, consider the following two methods:

```
void processData(Object obj){ }  
void processData(String str){ }
```

and the method call `processData(myString);`, where `myString` is a `String` variable. Since `String` is an `Object`, the `String` argument matches the parameter list of both the methods and so, both the methods are capable of accepting the method call. However, `String` is an **exact match** to the declared type of `myString` and so, the compiler will select the second method.

This rule applies to primitives as well. Thus, out of the following two methods, the first method is selected when you call `processData(10);` because `10` is an `int`, which matches exactly to the parameter type of the first method even though the long version of the method is also perfectly capable of accepting the value.

```
void processData(int value){ }  
void processData(long value){ }
```

3. **Most specific method** - If more than one method is capable of accepting a method call and none of them is an exact match, the one that is "**most specific**" is chosen by the compiler. For example, consider the following two methods:

```
void processData(Object obj){ }  
void processData(CharSequence str){ }
```

and the method call `processData("hello");`. Remember that `String` extends `CharSequence` and `CharSequence` extends `Object`. Thus, a `String` is a `CharSequence` and a `String` is also an `Object`. So here, neither of the methods has a parameter list that is an exact match to the type of the argument but both the methods are capable of accepting a `String`. However, between `Object` and `CharSequence`, `CharSequence` is more specific and so the compiler will select the second method.

It is important to understand what "**most specific**" means. It really just means closer or more similar to the type of the parameter being passed. A `String` is closer to a `CharSequence` than it is to an `Object`. Technically, a subclass (or a subtype) is always more specific than a super class (or supertype). If you have trouble understanding this, remember that `Cat` is more specific than `Animal`, `Apple` is more specific than `Fruit`.

Since primitives are not classes, there is no subclass/superclass kind of relation between them as such but Java does define the subtype relation for them explicitly, which is as follows:

```
double > float > long > int > char
and
int > short > byte
```

It means, `float` is a subtype of `double`, `long` is a subtype of `float`, `int` is a subtype of `long`, and `char` is subtype of `int`. And also, `short` is a subtype of `int` and `byte` is a subtype of `short`.

Based on the above, you can easily determine which of the following two methods will be picked if you call `processData(byteVar)`; (assume that `byteVar` is declared as `byte`).

```
void processData(int value){ }
void processData(short value){ }
```

The short version will be picked because `short` is a subtype of `int` and is therefore, more specific than an `int`.

Here is another interesting situation that can be explained easily based on the above subtype hierarchy of primitives. What will the following code print when compiled and run?

```
public class TestClass{
    public void m(char ch){
        System.out.println("in char");
    }
    public static void main(String[] args) {
        byte b = 10;
        new TestClass().m(b);
    }
}
```

The above code will cause a compilation error saying, "error: incompatible types: possible lossy conversion from byte to char". Observe that `char` is not a supertype of `byte`. Therefore, the method `m(char ch)` is not applicable for the method call `m(b)`. This makes sense because even though `char` is a larger data type than `byte`, `char` cannot store negative values, while `byte` can.

4. **Consider widening before autoboxing** - Since autoboxing only came into existence when Java 5 was released, it is necessary to give higher priority to the primitive versions if the argument can be widened to the method parameter type so that existing code will keep working as before. Therefore, out of the following two methods, the short version will be picked instead of the Byte version if you call `processData( (byte) b)`; even though the Byte version is an exact match with `byte` after autoboxing.

```
void processData(short value){ }  
  
void processData(Byte value){ }
```

5. **Consider autoboxing before varargs** - This rule mandates that if an argument can be autoboxed into a method parameter type then that method be considered even if a method with varargs of the same type is available. This explains why if you call `processData(10)`; then the Integer version (and not the `int...` version) is picked out of the following two.

```
void processData(int... values){ }  
  
void processData(Integer value){ }
```

Let us now try to figure out what happens if you call `processData((byte) 10)`; with the same two methods available.

```
void processData(int... values){ }  
  
void processData(Integer value){ }
```

Just apply the rules one by one:

1. **Applying rule 1** - Requires no application.
2. **Applying rule 2** - There is no method available whose parameter type matches exactly to `byte` because `byte` doesn't match exactly to `int...` i.e. `int[]` or `Integer`.
3. **Applying rule 3** - There is no method available whose parameter type matches to `byte` after widening a `byte` to any of the types a `byte` can be widened to (i.e. `short`, `int`, `long` etc.). Note that a `byte` cannot be widened to `int[]`.
4. **Applying rule 4** - There is no method available whose parameter type matches to `byte` after autoboxing it to `Byte`. Remember that a primitive type can only be autoboxed to the same wrapper type. So a `byte` can only be autoboxed to `Byte` and not to anything else such as `Short` or `Integer`.
5. **Applying rule 5** - A `byte` can be widened to `int` and an `int` can be accepted by a method that takes `int...`. Therefore, the varargs version of the method will be invoked

Let me show you another example. What will happen if we call `processData(10)`; with the following two methods available:

```
void processData(Long value){ }  
  
void processData(Long... values){ }
```

Observe that 10 is an `int`. Do we have a `processData` method that takes an `int`? No. So Rule 2 about exact match doesn't apply. `int` is not a subtype of `Long` or `Long[]` so Rule 3 doesn't apply either. Let's see if Rule 4 is of any help. 10 can be widened to a `long`, `float`, or `double` but we don't have any method that takes any of these types. Finally, as per Rule 5, 10 can be boxed into an `Integer` but that won't work either because there is no method that takes an `Integer`. Remember that `processData(Long )` cannot accept an `Integer` because `Integer` is not a subtype of `Long`.

Well, we don't have anymore options left to try. This means the compiler can't tie this call to any method and will therefore, raise an error saying, "Error: no suitable method found for `processData(int)`".

Method selection is not a trivial topic. The Java language specification spends a considerable number of pages defining the rules of method selection. Since it is easy to get bogged down with all those rules, I have tried to simplify them so they are easy to remember. You will be able to answer the questions in the exam and will also be able to figure out what a piece of code does in real life using the four points I have mentioned above.

Just be aware that there are several situations that cannot be explained by these points. Most of them involve Generics, a topic that is not on the exam. You should go through relevant sections of JLS if you are interested in exploring this topic further.

#### Exam Tip

You will most likely get only a single question on this topic. If you forget the above rules and find yourself taking too much time in figuring out the answer, I suggest you leave that question and move ahead. There is no point in wasting too much time on a question that is very easy to answer incorrectly.

## 11.3 Passing object references and primitive values into methods

### 11.3.1 Passing arguments to methods

This section requires that you have a clear understanding of the difference between an object and a reference. I suggest you to go through the section 3.6.5, "Relation between Class, Object, and Reference" and section 5.2, "Difference between reference variables and primitive variables" to refresh your memory before proceeding with this topic.

## Pass by value

To understand how Java passes arguments to methods, there is just one rule that you need to remember - Java uses **pass-by-value** semantics to pass arguments to methods. There are no exceptions. I mentioned this rule right at the beginning because if you keep this rule in your mind, this topic will feel like a piece of cake to you. You will never get confused with any code that the exam throws at you.

Let me start with the following simple code:

```
public class TestClass {

    public static void main(String[] args){
        int a = doubleIt(100);
        System.out.println(a); //prints 200
    }

    public static int doubleIt(int x){
        return 2*x;
    }

}
```

In the above code, the main method invoke the doubleIt method and passes the value 100 to the method. Just before starting the execution of the doubleIt method, the JVM initializes the parameter x with the value that was passed, i.e., 100. Thus, x now contains the value 100. The return statement returns the value generated by the statement 2\*x back to the caller. The JVM assigns this value to the variable a. Thus, a now contains 200. This is what is printed out.

Quite simple so far, right? Let me modify the code a bit now:

```
public class TestClass {

    public static void main(String[] args){
        int a = 100;
        int b = doubleIt(a);
        System.out.println(a+", "+b);
    }

    static int doubleIt(int x){
        return 2*x;
    }

}
```

The only thing that I have changed is that instead of passing the value 100 directly to the doubleIt method, I am passing the variable a. The JVM notices that the method call is using a variable as an argument. It takes the value contained in this variable (which is 100) and passes it to the method. Rest is exactly the same as before. Just before starting the execution of the doubleIt method, the JVM initializes the parameter x with the value that was passed, i.e., 100. Thus, x now contains

the value 100. The return statement returns the value generated by the statement  $2*x$  back to the caller. The JVM assigns this value to the variable `b`. Thus, `b` now contains 200. Finally, 100, 200 is printed.

Observe that `doubleIt` has no knowledge of the variable `a`. It only gets the value contained in the variable `a`, i.e., 100, which is assigned to `x`. Are you with me so far? Alright, here comes the twist. Take a look at the following code:

```
public class TestClass {

    public static void main(String[] args){
        int a = 100;
        int b = doubleIt(a);
        System.out.println(a+", "+b);
    }

    static int doubleIt(int x){
        x = 200;
        return 2*x;
    }
}
```

If you have understood the logic I explained earlier, you should be able to figure out what the above code prints. Let's follow the same process for analyzing what is happening here. The JVM notices that the method call is using a variable as an argument. It takes the value contained in this variable (which is 100) and passes it to the `doubleIt` method. Just before starting the execution of the `doubleIt` method, the JVM initializes the parameter `x` with the value that was passed, i.e., 100. Thus, `x` now contains the value 100. Next, `x` is assigned a new value 200 by the statement `x = 200`; In other words, the `x` is overwritten with a new value 200. The point to understand here is that this assignment has no effect of the variable `a` that was used in the calling method because `doubleIt` has absolutely no idea where the original value of 100 that it was passed as an argument came from. It merely gets the value 100 as an argument. In other words, the JVM passed only the value 100 to the method and not the variable `a`.

Next, the return statement returns the value generated by the statement  $2*x$  back to the caller, which is 400. The JVM assigns this value to the variable `b`. Thus, `b` now contains 400. At this time, you should realize that nothing was done to change the value of the variable `a` at all. It still contains the same value as before, i.e., 100. Therefore, the print statement prints 100, 400. The above example also shows that it is not possible for a method to change the value of the variable that was passed as an argument by the caller because that variable is never sent to the method. Only its value is sent. That is why the term "**pass by value**" is used to describe parameter passing in Java.

### 11.3.2 Passing objects to methods

In the previous lesson, you saw how passing a primitive variable to a method works. Passing a reference variable works exactly the same way. Check out the following code:

```
class Data{
```

```

    int value = 100;
}

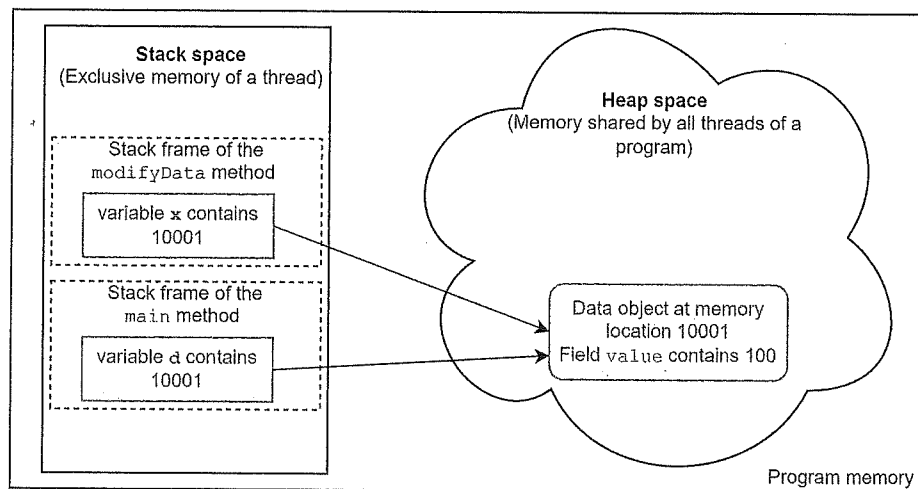
public class TestClass {

    public static void main(String[] args){
        Data d = new Data();
        modifyData(d);
        System.out.println(d.value); //prints 200
    }

    public static void modifyData(Data x){
        x.value = 2*x.value;
    }
}

```

In the above code, the main method creates a Data object and saves its reference in the variable d. It then calls the modifyData method. During execution, the JVM notices that the call to modifyData method uses a reference variable. Recall our discussion on reference variables in "Working with data types" chapter that a reference variable doesn't actually hold an object. It just holds the address of the memory location where the object resides. While invoking the modifyData method, the JVM copies the value stored in the variable d into the method parameter x. So, for example, if the Data object is stored at the memory address 10001, the variable d contains 10001 and the JVM copies this value into the variable x. Thus, the variable x also now contains the same address as d and thus, in a manner of saying, variable x also starts pointing to the same object as the one pointed to by the variable d. This situation is illustrated in figure below.



Passing an object to a method 1

The point to note here is that even though it looks as if we passed the Data object created

in `main` to `modifyData`, all we actually passed was the value stored in variable `d` (which was nothing but the address of the `Data` object) to `modifyData`. The `Data` object itself remained exactly at the same memory location where it was. We didn't move it, pass it, or copy it, to anywhere.

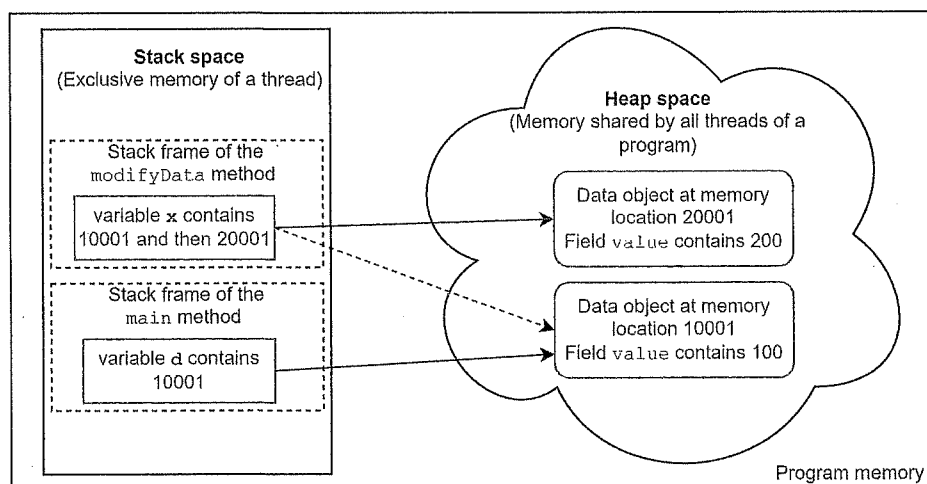
The `modifyData` method uses the reference variable `x` to modify the `value` field of `Data` object pointed to by `x` to 200. When the control goes back to the main method, it prints `d.value`, which is now 200.

Let us make a small modification to the `modifyData` method of the above code:

```
public static void modifyData(Data x){
    x = new Data();
    x.value = 2*x.value;
}
```

All we have done is inserted the statement `x = new Data();`. Can you tell what the main method will print now?

When the JVM invokes `modifyData` method, it makes `x` point to the same `Data` object as the one pointed to by variable `d`. This part is the same as before. But the first statement of the updated method creates a new `Data` object and makes `x` point to this new object. For example, if this new `Data` object resides at address 20001, then `x` now contains 20001 (instead of 10001). The next statement updates the `value` field using the variable `x`. Since `x` now points to the new `Data` object, this update statement updates the `value` field of the new `Data` object instead of the old `Data` object. When the control goes back to the main method, it prints `d.value`. This prints 100 because we never modified the variable `d` and so, `d` still points to the original `Data` object. This is illustrated in figure below.



Passing an object to a method 2

In the two examples above, we passed a variable (the local variable `d`) as an argument to the `modifyData` method. Now, take a look at the following code:

```
class Data{
    int value = 100;
}

public class TestClass {

    public static void main(String[] args){
        modifyData(new Data());
    }

    public static void modifyData(Data x){
        x.value = 2*x.value;
    }
}
```

Observe the statement `modifyData(new Data());`. We are not passing any variable to the `modifyData` method here. So, are we really passing the `Data` object directly to the method? No, although we do not have any explicit variable to store the address of the `Data` object, the compiler creates a temporary reference variable implicitly. It creates the `Data` object in the heap space and assigns it to this temporary variable. While making the method call, it is the value of this variable that is passed to the `modifyData` method. Of course, since we have not saved a reference to this `Data` object in the main method, we won't be able to refer to this object after the `modifyData` method returns. This technique is used all the time while printing messages to the console using the `print/println` method. For example, when you do `System.out.println("hello world");` the compiler passes the value of the temporary variable that points to the `String` object containing "hello world" to the `println` method.

### 11.3.3 Returning a value from a method

Just like parameters, returning a value from a method also uses "pass by value" semantics.

In the case of primitives, a return statement can return either the value directly (e.g. `return 100;` `return true;` `return 'a';` etc.) or the value of a variable (e.g. `return i;` where `i` is an `int` variable).

In the case of objects, a return statement can either return the value of an explicit variable (e.g. `return str;` where `str` is a `String` variable) or the value of an implicit temporary variable that references the object (e.g. `return "hello";` `return new Student();` etc.). In both cases, it is really the address where the object is stored that is returned.

What the caller does with the return value is irrelevant to the method that returns a value. The caller can either use the return value further in the code or ignore the return value altogether.

## 11.4 Apply the static keyword to methods and fields

### 11.4.1 Apply the static keyword to methods and fields

I have already discussed the meaning of the word `static` and what it implies when applied to a method or a field in the "Kickstarter for Beginners" chapters.

In this section, I will dig a little deeper and explain the nuances of `static` from the perspective of the exam. Let us start with the syntax. You can declare any member of a class (i.e. a method, a field, or any nested type definition) as `static` using the keyword `static`. In the case of a method or a field, this keyword must appear before the return type of the method or the type of the field respectively. For example:

```
class ConnectionHelper{
    static int idle_timeout;
    static String url;
    private static void ping(){ } //order of modifiers doesn't matter
    static public final void check(){ } //order of modifiers doesn't matter
}
```

### 11.4.2 Accessing static members

A static member exists as a member of the owning class and not as a member of an instance of the owning class. In other words, a static member does not require an instance of the owning class to exist. A static member, therefore, can be accessed by specifying the name of the owning class. For example, the static members of `ConnectionHelper` and `Request` can be accessed from another class like this:

```
class TestClass{
    public static void main(String[] args){
        System.out.println(ConnectionHelper.idle_timeout); //prints 0 (why?)
        System.out.println(ConnectionHelper.url); //prints null (why?)
        Request.Data rc = new Request.Data();
    }
}
```

Accessing a static member using the name of the owning class is the standard and recommended way of accessing static members. However, Java allows a static member to be accessed through a variable as well. For example, the static variable `idle_timeout` of class `ConnectionHelper` can also be accessed like this:

```
class TestClass{
    public static void main(String[] args){
        ConnectionHelper c = null;
        System.out.println(c.idle_timeout); //prints ConnectionHelper's idle_timeout
    }
}
```

Observe that `c` is null. But the compiler doesn't care about `c` being null because it notices that `idle_timeout` is a static variable and an instance of `ConnectionHelper` is not needed to access `idle_timeout`. The compiler knows what `idle_timeout` in the statement `c.idle_timeout` implies and effectively translates it to `ConnectionHelper.idle_timeout`.

This simple example highlights an important aspect of accessing static members: access to static members is decided by the compiler at compile time by checking the declared type of the variable. It is not decided by the JVM at run time. The compiler knows that the type of the variable `c` is `ConnectionHelper` and that `idle_timeout` is a static member of `ConnectionHelper`, that is why the compiler binds the call to `c.idle_timeout` to `ConnectionHelper`'s `idle_timeout`. The compiler doesn't care what `c` may point to at run time. Now, armed with this knowledge, can you tell what the following code will do?

```
class TestClass{
    public static void main(String[] args){
        ConnectionHelper c = null;
        c.ping();
    }
}
```

That's right. It will compile and run fine (of course, without any output). But the interesting thing is that there will be no `NullPointerException`. Since the type of the reference variable `c` is `ConnectionHelper` and `ping()` is a static method of `ConnectionHelper`, the compiler binds the call to `c.pint()` to `ConnectionHelper`'s `ping()`.

This is called **static binding** or **compile time binding** because the compiler doesn't leave the decision of binding a call to the JVM. This concept will play an important role when we discuss inheritance and polymorphism later.

### 11.4.3 Accessing static members from the same class

A static member of a class can also be accessed directly from static as well as instance members of the same class without the need to use the class name. For example, the following class makes use of a static variable to count the number of instances of that class:

```
class InstanceCounter{
    static int count;
    InstanceCounter(){
        //directly accessing count from a constructor
        count++;
    }

    static void printCount(){
        //directly accessing count from a static method
        System.out.println(count);
    }

    void reduceCount(){
```

```
        //directly accessing count from an instance method
        count--;
    }
}
```

The following class shows various ways in which you can access static members of a class from another class:

```
class TestClass {
    public static void main(String[] args){
        InstanceCounter ic = new InstanceCounter();
        ic.printCount(); //accessing static method through a reference to an instance

        new InstanceCounter().printCount(); //accessing static method through an implicit
        reference to an instance

        InstanceCounter.printCount(); //accessing static method using the class name
        System.out.println(InstanceCounter.count); //accessing static field using the
        class name
    }
}
```

You should be able to tell what the above code will print.

#### 11.4.4 Accessing instance members from a static method

Since a static method belongs to a class and not to an object of that class, a static method does not execute within the context of any instance of that class. On the other hand, an instance method is always invoked on a specific instance of a class and so, it executes within the context of the instance upon which it is invoked. An instance method can access this instance using the **implicit** variable **"this"**. Since there is no instance associated with a static method, the variable **this** is not available in a static method.

The reason why the **"this"** variable is called an implicit variable becomes important here. This variable is not declared explicitly anywhere and the compiler assumes its existence in an instance method. Whenever the compiler sees an instance member being accessed from within a method directly, the compiler uses the **this** variable to access that member even when you don't type it explicitly in your code. However, when a static method tries to access an instance member, **this** is not available and so, the compiler complains that a non-static variable cannot be referenced from a static context.

The following code illustrates this point:

```
class Book{
    int name;
    static void printName1(){
```

```
        System.out.println(this.name); //will not compile
        System.out.println(name); //same as above. will not compile
    }
    void printName2(){
        System.out.println(this.name); //this is fine
        System.out.println(name); //same as above. this is fine
    }
}
```

In the above code, the compiler realizes that `name` is an instance variable and so, tries to access it through `this`, i.e., `this.name`. But since `this` is not available in `printName1`, it generates an error. There is no issue with `printName2` because `printName2` is an instance method and `this` is available in an instance method.

Remember that you don't always need to explicitly use the variable `this` to access instance members. You need to use it only if there is also a local variable with the same name declared in the method and you want to refer to the instance variable instead of the local one. Technically, `this` is used to "unshadow" an instance variable if it is shadowed by a local variable of the same name.

A common misunderstanding amongst beginners is that a static method cannot access instance fields of a class. This misunderstanding exists because they see or hear this statement in many places. However, it is an incomplete statement. The correct statement is that a static method cannot access an instance member without specifying the instance whose member it wants to access. It will be clear when you see the following code:

```
class Book{
    int name;
    static void printName(){
        Book b1 = new Book();
        Book b2 = new Book();
        System.out.println(b1.name); //this will compile fine
    }
}
```

In the above code, we are accessing the instance variable `name` from within a static method. Notice that there are two instances of `Book`. Each instance has its own copy of the variable `name`. But because we are using the reference `b1` to access `name`, the compiler knows exactly whose `name` we intend to access. It knows that we want to access the `name` field of the `Book` instance pointed to by the reference variable `b1`. That is why there is no problem. This shows that an instance field can indeed be accessed from a static method as long as we specify exactly the instance whose field we want to access.

## 11.5 Exercise

1. Create a method named **add** that can accept any number of `int` values and returns the sum of those values.

2. Create another method named **add** in the same class that can accept any number of `int` values but returns a `String` containing concatenation of all those values. What can you do to resolve compilation error due to the presence of the two methods with same signature? Invoke these methods from the main method of this class.
3. Create a class named **Student** with a few fields such as `studentId`, `name`, and `address`. Should these fields be static or non-static? Add the main method to this class and access the fields from the main method.
4. Add a static field to **Student** class. Access this field from another class. Use appropriate import statement to access the field directly. Change accessibility of the field and see its impact on the code that tries to access it.
5. Create a method named `method1` in **TestClass** that accepts a `Student` object and updates the static as well as instance fields of this object. Pass the same `Student` object to another method named `method2` and print the values. Assign a new `Student` object to the `Student` variable of `method2` and set its fields to different values. After returning back to `method1`, print the values again. Explain the output.
6. Add a constructor in `Student` class that accepts values for all of its instance fields. Add a no-args constructor in `Student` class that makes use of the first constructor to set all its instance fields to dummy values.
7. Create a class named **Course** in different package. Add a static method named **enroll** in this class that accepts a `Student`. Use different access modifiers for fields of `Student` class and try to access them from the `enroll` method.